SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 2 QUESTIONS

Name:N.AKSHAYA
Reg.no:192425129
Course code:CSA5101
Course name: Cryptography and Network Security
Date: 2-11-2026

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.

- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks) Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

ANSWER

1.1 Comparison of symmetric algorithms

Algorithm	Structure and key features	Evaluation
DES	Feistel block cipher, 64-bit block, 56-bit effective key, 16 rounds using expansion, key mixing, S-box substitution and permutation.	Obsolete; the short key can be brute-forced. Do not use for new healthcare data.
3DES	Applies DES three times, usually in EDE form, to a 64-bit block. Uses two or three DES keys.	Stronger than single DES but slow and has a small block size. Legacy use only.
Blowfish	Feistel cipher, 64-bit block, variable key length of 32–448 bits and 16 rounds with key-dependent S-boxes.	Fast historically, but the 64-bit block size limits safe use for large data.
RC5	Parameterized block cipher. RC5-w/r/b defines word size, rounds and key bytes. Uses XOR, modular addition and data-dependent rotations.	Flexible, but security depends on parameter choices and implementation.

A large key space alone does not guarantee security. Block size, round design, known attacks, implementation and mode of operation must also be considered.

1.2 Block and stream ciphers

A block cipher encrypts fixed-size blocks and uses a mode to process long messages. A stream cipher generates a keystream and combines it with plaintext, usually using XOR. Block ciphers are suitable for files and stored records; stream constructions are useful for continuous or low-latency data. Nonce rules must be followed carefully.

1.3 Modes of operation

Mode	Operation and requirements	Main properties
ECB	Encrypts every block independently; no IV.	Identical plaintext blocks produce identical ciphertext blocks. Parallel, but leaks patterns. Avoid.
CBC	XORs each plaintext block with the previous ciphertext; first block uses an unpredictable IV.	Encryption is sequential. A bit error affects the current and following block portion.
CFB	Encrypts the previous ciphertext/IV and XORs the result with plaintext.	Sequential encryption; suitable for smaller segments. Requires a unique IV.
OFB	Generates a keystream by repeatedly encrypting feedback state.	A bit error flips only the corresponding plaintext bit. IV must not be reused with the same key.
CTR	Encrypts nonce-plus-counter values to create a keystream.	Fast and parallel. A nonce/counter must never repeat under the same key.

1.4 Pattern leakage and nonce reuse

If two plaintext blocks are identical, ECB produces identical ciphertext blocks under the same key. This reveals repeated patterns. In CTR, ciphertext is C = P XOR S, where S is the keystream. If the same nonce and key are reused, C1 XOR C2 = P1 XOR P2, which can expose information about both messages. A unique nonce is therefore essential.

1.5 Operation count and recommendation

For N bytes and block size B, approximately ceil(N/B) blocks are processed. ECB, CBC and CTR each need roughly one block-cipher operation per block; 3DES performs three DES transformations per block and is slower. Actual time depends on the processor and library.

Recommendation: Do not use DES or 3DES for new records. Blowfish and RC5 may be studied but are not preferred for a new system. Use AES-GCM with a unique nonce and authentication-tag verification for bulk healthcare data. Use a trusted library.

Experimental evidence to add: measure encryption and decryption time for 1 KB, 100 KB, 1 MB and 10 MB files. Record the algorithm, mode, operation count and measured time; do not invent results.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition $\gcd(e, \varphi(n)) = 1$.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

ANSWER

2.1 Number-theory operations

The greatest common divisor gcd(a,b) is the largest integer dividing both values. Euclid's algorithm repeatedly replaces (a,b) with (b, a mod b) until b is zero. The Extended Euclidean Algorithm finds x and y such that ax + by = gcd(a,b). If gcd(a,m)=1, the modular inverse of a is x mod m.

Fast modular exponentiation calculates a^e mod m using square-and-multiply. It processes the exponent bit by bit and requires O(log e) multiplication steps, unlike naive repeated multiplication.

2.2 Pseudocode

GCD(a,b): while b != 0: (a,b) = (b,a mod b); return a

EXTENDED_EUCLID(a,b): if b=0 return (a,1,0); otherwise recursively find (g,x1,y1) for (b,a mod b), then return (g,y1,x1-floor(a/b)*y1)

MOD_INVERSE(a,m): find (g,x,y)=EXTENDED_EUCLID(a,m); if g != 1, no inverse exists; otherwise return x mod m

MOD_POW(a,e,m): result=1; while e>0, if e is odd set result=result*a mod m; set a=a*a mod m; set e=floor(e/2); return result

2.3 Euler's totient and RSA

Euler's totient φ(n) counts integers from 1 to n that are relatively prime to n. For distinct primes p and q, φ(pq)=(p−1)(q−1). RSA selects e such that gcd(e,φ(n))=1. This guarantees a modular inverse d satisfying ed ≡ 1 (mod φ(n)).

2.4 Prime testing

Method	Working and limitation
Trial division	Tests divisors up to √n. Reliable for small numbers but slow for large candidates.
Fermat test	Checks a^(n−1) ≡ 1 (mod n). Fast, but some composites pass.
Miller–Rabin	Uses n−1=2^s d and modular exponentiation for selected bases. Efficient; repeated rounds make false acceptance very unlikely.

Carmichael numbers such as 561, 1105 and 1729 are composite but can pass Fermat tests for many bases. Miller–Rabin is therefore preferred for practical large-prime testing.

2.5 Prime density and engineering decision

The prime number theorem estimates that a large integer near x is prime with probability about 1/ln(x). Larger candidates require more attempts on average, but the growth is manageable with efficient testing. Use a cryptographically secure random generator. 1024-bit RSA is no longer recommended; 2048-bit is a common minimum baseline and 3072-bit provides a larger margin. Use a vetted library for real key generation.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

ANSWER

3.1 RSA key generation

Choose two distinct large random primes p and q.

Calculate $n=pq$ and $\varphi(n)=(p-1)(q-1)$.

Choose e such that $\gcd(e,\varphi(n))=1$.

Calculate $d=e^{-1} \bmod \varphi(n)$, so $ed \equiv 1 \pmod{\varphi(n)}$.

The public key is (n,e); the private key is (n,d).

Encryption: $C=M^e \bmod n$. Decryption: $M=C^d \bmod n$. Real RSA encryption must use secure padding such as OAEP.

3.2 Hand-checkable example

Let $p=5$ and $q=11$. Then $n=55$ and $\varphi(n)=40$. Choose $e=3$ and $d=27$ because $3\times27=81 \equiv 1 \pmod{40}$. For $M=7$, $C=7^3 \bmod 55=13$. Decryption gives $13^{27} \bmod 55=7$. This verifies the arithmetic only; such small values are insecure.

3.3 Weaknesses and attacks

- Textbook RSA is deterministic: the same message and key produce the same ciphertext, revealing repeated messages.
- Small messages and small public exponents can be vulnerable when padding is absent.
- An invalid choice with $\gcd(e,\varphi(n)) \neq 1$ means the required private exponent does not exist.
- If an attacker factors n into p and q, the attacker can calculate $\varphi(n)$ and recover d.
- Weak random primes, shared prime factors, exposed private keys and side-channel leakage can defeat RSA even with a large modulus.

3.4 Factoring and OAEP

Trial division tests candidate factors up to $\sqrt{n}$. Fermat factorisation writes $n=a^2-b^2=(a-b)(a+b)$, and is useful when the two factors are close. These attacks are practical only for small classroom numbers. RSA-OAEP adds randomized encoding, so repeated encryption of the same message normally produces different ciphertexts and textbook-RSA patterns are avoided. Use a trusted library and never create custom padding.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from $(g^a)^b \equiv (g^b)^a \pmod{p}$.

- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

ANSWER

4.1 Diffie–Hellman

The public parameters are a prime p and generator g. Clinic A chooses secret a and sends A=g^a mod p. Clinic B chooses secret b and sends B=g^b mod p. A computes K=B^a mod p and B computes K=A^b mod p. Both obtain g^(ab) mod p.

4.2 Pseudocode and example

Public: p,g. A chooses a and sends A=MOD_POW(g,a,p). B chooses b and sends B=MOD_POW(g,b,p). A computes MOD_POW(B,a,p); B computes MOD_POW(A,b,p).

Example: p=23, g=5, a=6 and b=15. A=5^6 mod 23=8; B=5^{15} mod 23=19. A calculates 19^6 mod 23=2, and B calculates 8^{15} mod 23=2. Both obtain the same key value. These toy parameters are not secure.

4.3 Brute force and man-in-the-middle attack

An attacker may try to recover a from A=g^a mod p by solving a discrete logarithm. This is easy for small toy values but infeasible for approved large groups with classical computers. Unauthenticated Diffie–Hellman is vulnerable to a man-in-the-middle attack: an attacker replaces the public values and creates one key with each clinic, allowing traffic to be read and re-encrypted.

Prevent this by authenticating the key exchange with digital signatures and certificates. Each clinic must validate the peer identity, certificate chain, validity, key usage and revocation status, and verify the signature over the handshake.

4.4 Key-management policy

1. Generate keys with a cryptographically secure random-number generator.
2. Protect private keys in an HSM or secure keystore and restrict access by role.
3. Distribute public keys and certificates through authenticated channels; never send private keys.
4. Set key and certificate lifetimes and rotate them regularly or after suspected compromise.
5. Revoke expired or compromised certificates and reject revoked identities.
6. After compromise, revoke the key, investigate, generate new keys and re-establish sessions.
7. Use separate short-lived session keys and erase them after use.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks) Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

ANSWER

5.1 ECC concept and key agreement

ECC uses points on a curve such as $y^2=x^3+ax+b$ over a finite field. A private key is an integer and the public key is a point formed by multiplying a base point G by that integer. A chooses a and publishes PA=aG; B chooses b and publishes PB=bG. Both compute abG and obtain the same shared point. Real systems must use standard curves and trusted libraries.

5.2 RSA, DH and ECC comparison

Method	Security basis and use	Practical points
RSA	Integer factorisation; encryption and signatures.	Large keys and signatures; widely supported but heavier.
Finite-field DH	Discrete logarithm; shared-key agreement.	Larger public values; must be authenticated to prevent MITM.
ECC (ECDH/ECDSA)	Elliptic-curve discrete logarithm; key agreement and signatures.	Smaller keys and communication overhead at comparable classical security; suitable for constrained clinics.

ECC is useful for low-resource clinics because it provides strong classical security with smaller public keys and less communication overhead. Use authenticated ECDH with certificates, secure random generation, approved curves and a modern symmetric cipher for patient records. ECC alone does not prevent a man-in-the-middle attack.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks) Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk. **Evidence Expected:** Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

ANSWER

6.1 Digital signature process

The sender calculates a hash of the document, for example SHA-256.

The sender signs the hash using the private signing key.

The sender sends the document, signature and certificate.

The receiver hashes the document and verifies the signature using the authenticated public key.

A valid signature supports integrity, sender authentication and non-repudiation when identity and key management are reliable. A changed document should cause verification to fail.

6.2 Signature algorithms

Algorithm	Key idea	Important issue
RSA-PSS	RSA signature using secure randomized encoding.	Requires correct padding and secure implementation.
DSA/DSS	Finite-field digital signature.	Nonce must be secret and unique; reuse may reveal the private key.
ECDSA	Elliptic-curve digital signature.	Nonce reuse or bias can expose the private key; deterministic nonce generation can reduce random-number risk.

6.3 Hybrid encryption

Public-key encryption is inefficient for large files. Hybrid encryption uses a random symmetric session key for the file and public-key cryptography to establish or protect that key.

Authenticate the receiver's certificate/public key.

Generate a fresh session key and unique nonce.

Encrypt the file with AES-GCM.

Establish the session key using authenticated ECDHE, or wrap it with RSA-OAEP when required.

Sign important file metadata or the transaction package.

The receiver verifies the certificate and signature, establishes the key, verifies the authentication tag and decrypts the file.

The receiver must reject a file if its signature or authentication tag fails. Direct RSA is unsuitable for large files because of message-size limits and high processing cost. Hybrid encryption performs a small number of public-key operations and fast symmetric encryption over the full file.

6.4 Tampering and quantum readiness

- Changing one byte of AES-GCM ciphertext causes authentication-tag verification to fail.
- Changing a digital signature causes signature verification to fail.
- Fresh nonces, session identifiers and protocol anti-replay controls help reject replayed exchanges.
- QKD uses quantum states to establish key material and can reveal eavesdropping through error rates, but needs an authenticated classical channel and special equipment.

- Shor's algorithm threatens RSA and conventional ECC/DH on a sufficiently powerful quantum computer. Grover's algorithm gives a quadratic speed-up against symmetric-key search.

Migration plan: inventory cryptographic systems, design crypto-agile software, follow approved post-quantum standards, test post-quantum key establishment and signatures, and migrate long-lived sensitive records first. Standardised hybrid classical/post-quantum key establishment can be used during transition. QKD does not replace certificates, endpoint security or key management.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

ANSWER

A. Diagnose the bottleneck or weakness

The major limitation is using public-key encryption for bulk files or using unauthenticated Diffie–Hellman. Public-key operations are much slower than symmetric encryption, and RSA cannot encrypt arbitrarily large files. Unauthenticated DH permits a man-in-the-middle attacker to create separate keys with both clinics.

B. Design objective

Use public-key operations only a small number of times per session, encrypt bulk records with authenticated symmetric encryption, reject modified data and prevent unauthenticated clinics from joining.

C. Revised strategy

Give each hospital and clinic an identity certificate. Protect private keys in an HSM or secure keystore.

Use authenticated ECDHE and validate certificates and signatures during the handshake.

Derive separate session keys using a standard key-derivation function.

Encrypt patient records and large files with AES-GCM and a unique nonce for every encryption under a key.

Digitally sign important reports, prescriptions and transaction metadata where proof of origin is needed.

Rotate keys, revoke compromised certificates, monitor failures and maintain secure audit logs.

The design uses efficient symmetric encryption for large data and ECDHE for shared-secret establishment. It reduces repeated public-key operations to a small number per session. The trade-offs are certificate infrastructure, nonce management, secure random generation and key storage.

D. Quantum readiness

A sufficiently capable quantum computer using Shor's algorithm could break RSA and conventional ECC/DH. The network should inventory cryptographic dependencies, use crypto-agile designs and plan migration to approved post-quantum algorithms. QKD may help distribute keys on selected links but cannot replace authentication, endpoint protection or key management.

E. Prediction and validation

Prediction: hybrid encryption will be faster for large files and require far fewer public-key operations than direct RSA. An authenticated handshake will reject substituted public values. Modified ciphertext or signatures must fail verification.

Test	Expected result in revised design
Large file transfer	One session key establishment followed by fast symmetric encryption.
Repeated plaintext blocks	No ECB-style repeated-block pattern with a correctly used modern mode.
MITM attempt	Certificate/signature validation rejects substituted values.
File tampering	AEAD tag verification fails and plaintext is rejected.
Signature tampering	Signature verification fails.
Nonce reuse	Nonce uniqueness controls prevent reuse under the same key.

Measure 1 KB, 100 KB, 1 MB and 10 MB files on the student's computer. Record actual hybrid encryption time and public-key operation count. Do not invent timing values; attach the code and raw output.

F. Final design defence

The hybrid architecture is preferred because it protects large files efficiently, authenticates clinics, detects tampering and reduces communication overhead. Certificates and central key management support new clinic enrolment, key rotation and revocation. ECC is suitable for constrained devices.

Residual risks include compromised endpoints, stolen keys, poor random generation, nonce reuse, certificate errors, denial-of-service and future quantum threats. Reduce these risks through endpoint hardening, HSM storage, monitoring, secure updates, recovery procedures and post-quantum migration.

Final decision: use authenticated ECDHE, certificate-based identity, AES-GCM for bulk records, digital signatures for important transactions, central key lifecycle management and a planned post-quantum transition.

Conclusion

A secure healthcare network should use the correct cryptographic method for each task. AES-GCM protects large records efficiently, public-key methods establish session keys and authenticate clinics, digital signatures provide integrity and origin verification, and key management protects the complete system. A hybrid and crypto-agile design provides a practical balance of security, performance and future readiness.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7

Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis • Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.